

Beyond Message Passing: Expressive GNN Models

Jiaxuan You

Assistant Professor at UIUC CDS



CS512: Data Mining Principles, 2025 Fall

<https://ulab-uiuc.github.io/CS512/>

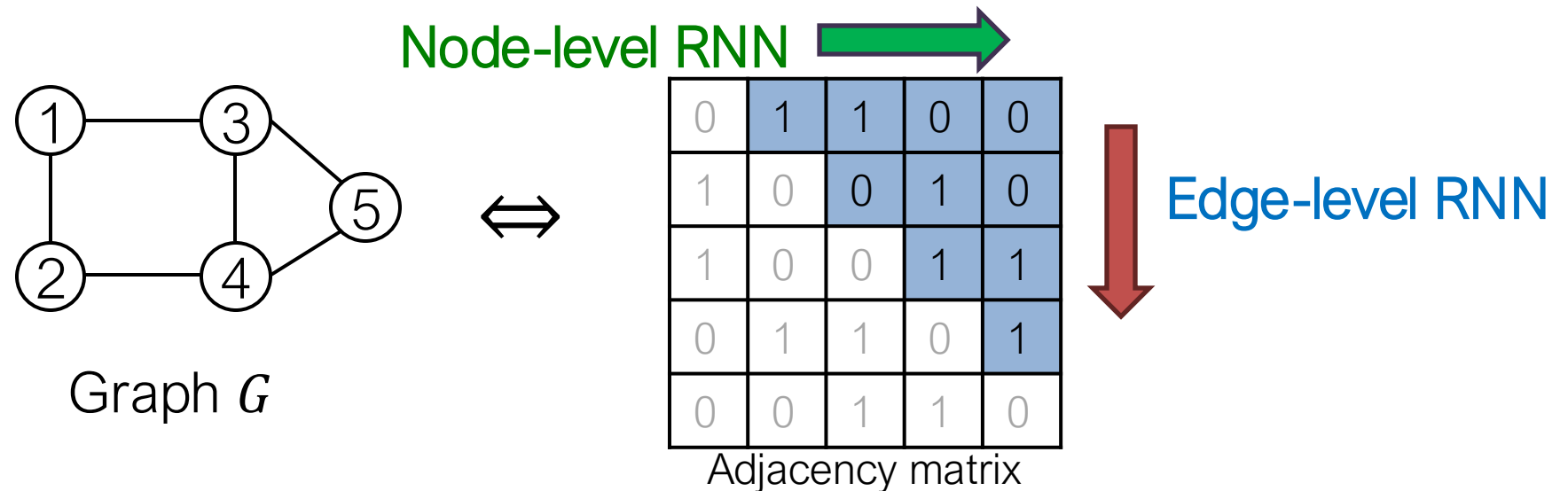
Reminder: Coding and Proposal Task

- **Group Idea proposal task** concluded
 - We will provide brief feedback to your proposal by this week
 - Get started with the project implementation
- **Coding Assignment 3** released, due **Oct 31 11:59 PM**
 - Submit your code and written answers downloaded from Colab to Canvas

Recap: GraphRNN: Two levels of RNN

Quick Summary of GraphRNN:

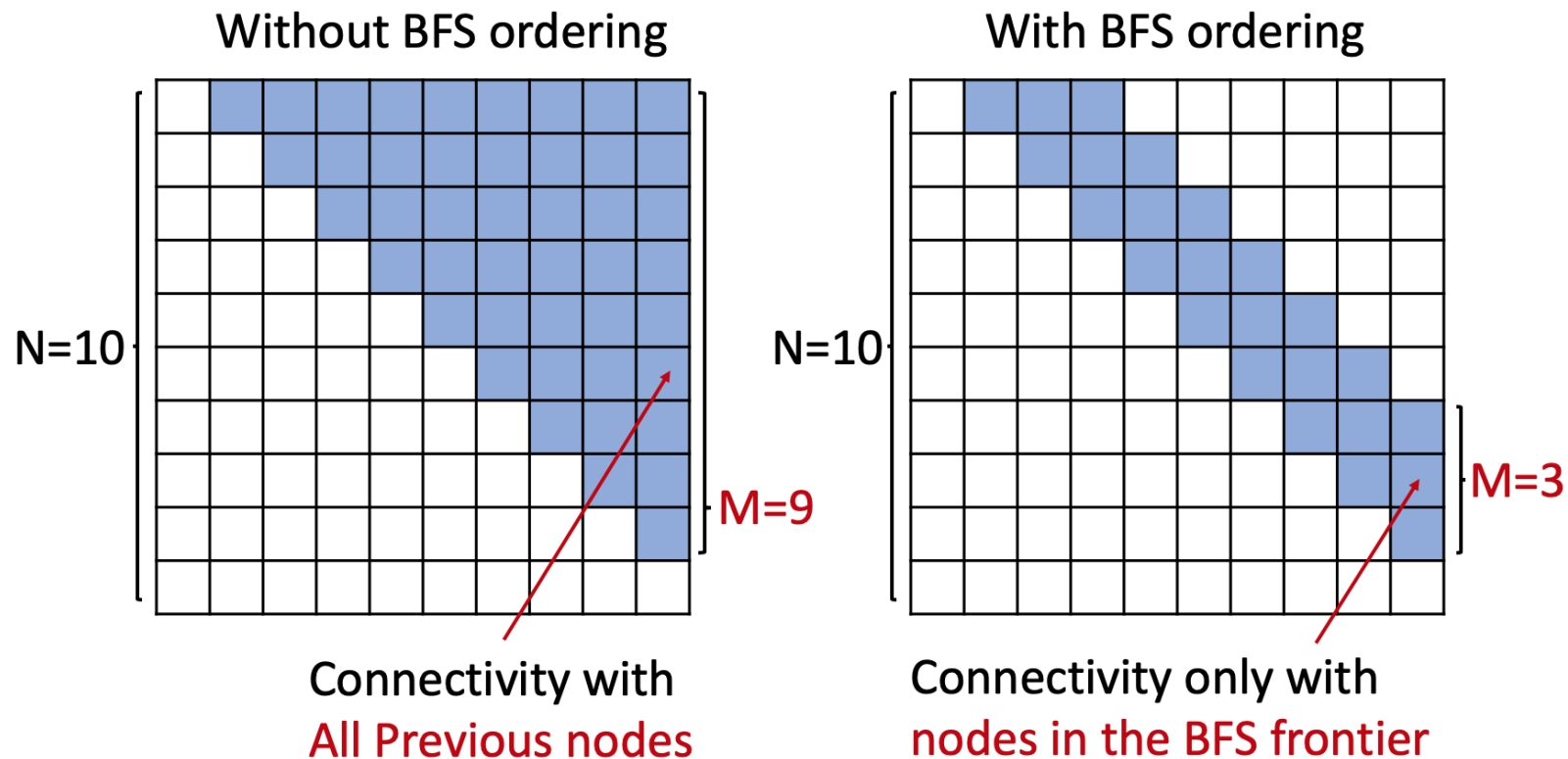
- Generate a graph by generating a two-level sequence
- Use RNN to generate the sequences
- **Next:** Making GraphRNN tractable, proper evaluation



Recap: GraphRNN Tractability via BFS

- **BFS reduces the number of steps for edge generation**

Adjacency matrices

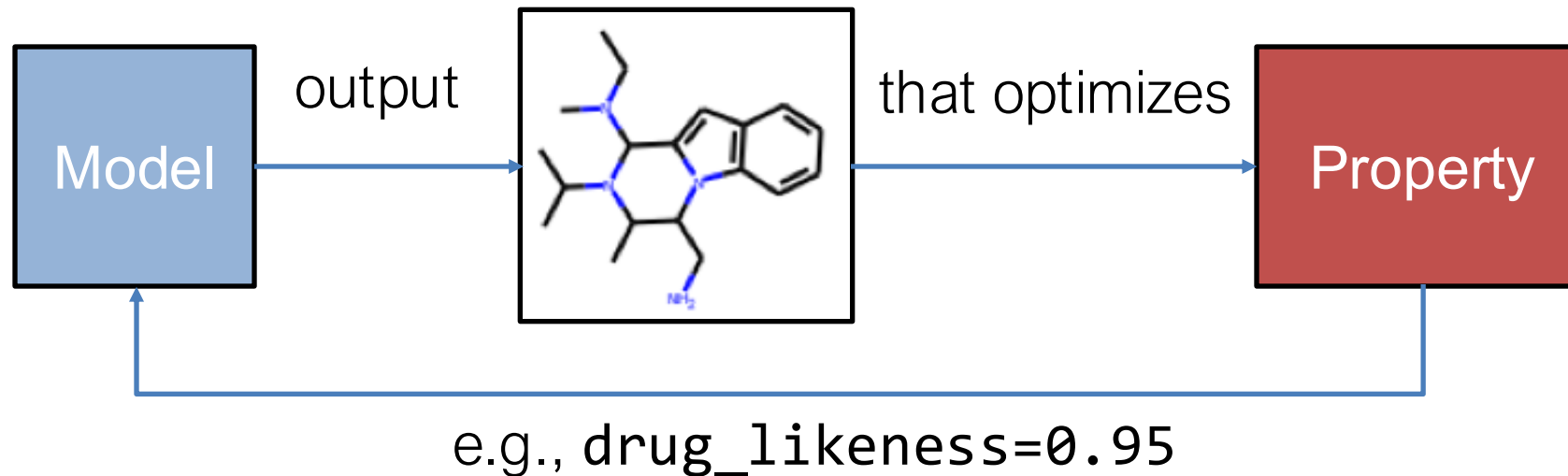


Beyond Prediction: Graph Generative Models

Application of Deep Graph Generative Models to Molecule Generation

Application: Drug Discovery

- **Question:** Can we learn a model that can generate **valid** and **realistic** molecules with **optimized** property scores?



Goal-Directed Graph Generation

Generating graphs that:

- **Optimize a given objective** (High scores)
 - e.g., drug-likeness
- **Obey underlying rules** (Valid)
 - e.g., chemical validity rules
- **Are learned from examples** (Realistic)
 - Imitating a molecule graph dataset
 - We have just covered this part

Goal-Directed Graph Generation

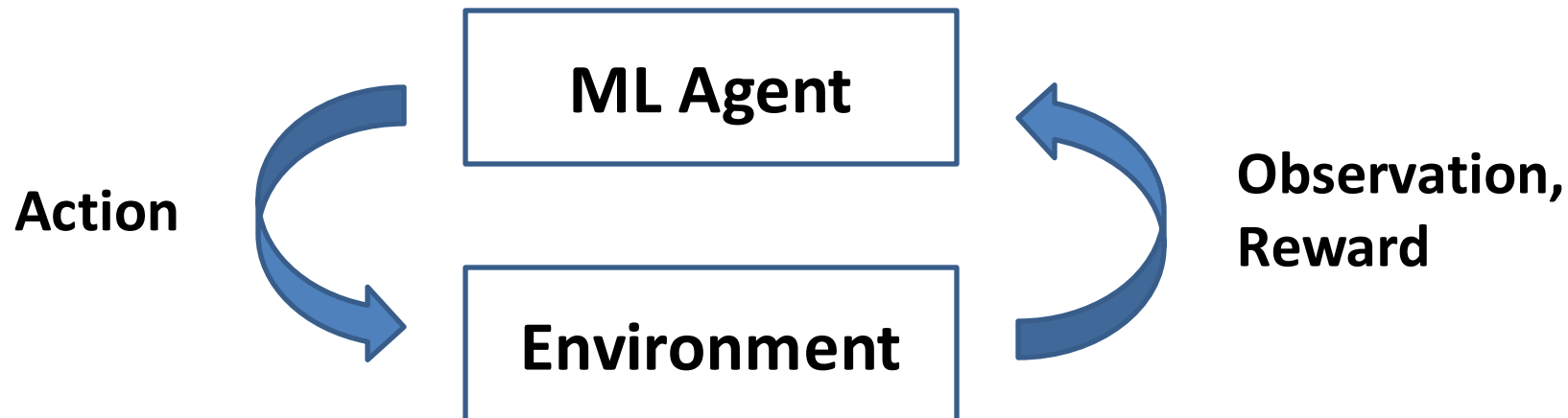
Generating graphs that:

- **Optimize a given objective** (High scores)
 - e.g., drug-likeness
- **Obey underlying rules** (Valid)
 - e.g., chemical validity rules
- **Are learned from examples** (Realistic)
 - Imitating a molecule graph dataset
 - We have just covered this part

A “Black-box” to Graph Generation:
Objectives like drug-likeness are governed by physical law

Idea: Reinforcement Learning

- A ML agent **observes** the environment, takes an **action** to interact with the environment, and receives positive or negative **reward**
- The agent then **learns from this loop**
- **Key idea:** Agent can directly learn from environment, which is a **blackbox** to the agent



Solution: GCPN

Graph Convolutional Policy Network (GCPN) combines **graph representation + RL**

Key component of GCPN:

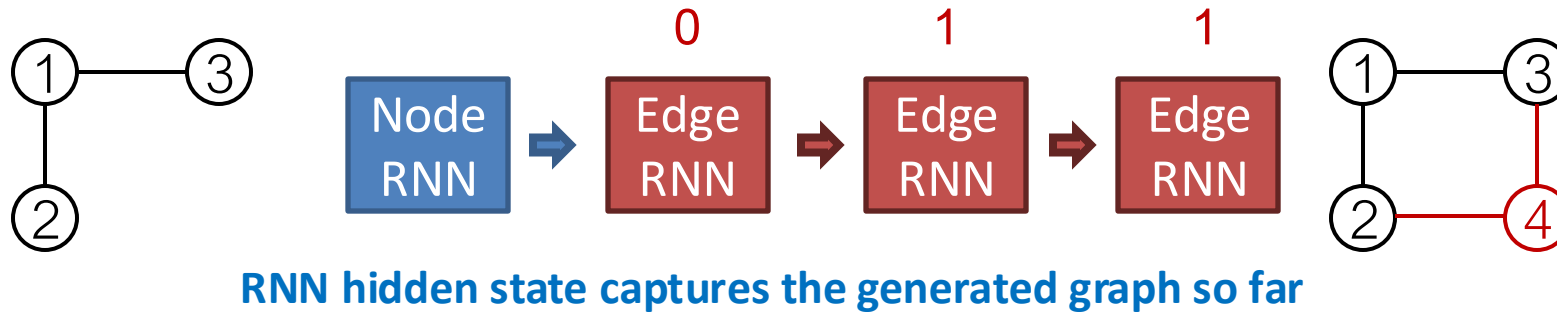
- **Graph Neural Network** captures graph structural information
- **Reinforcement learning** guides the generation towards the desired objectives
- **Supervised training** imitates examples in given datasets

GCPN vs. GraphRNN

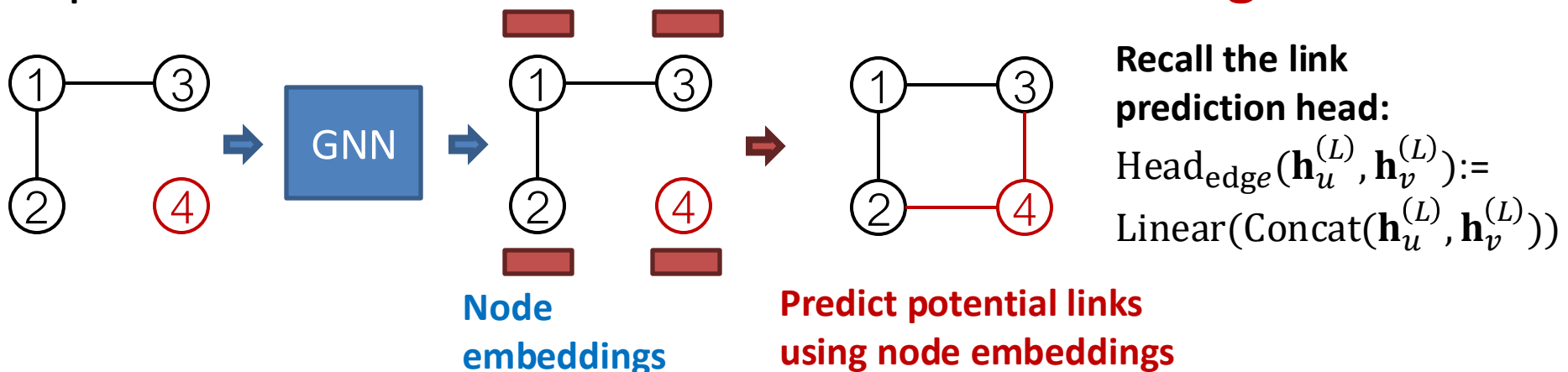
- **Commonality of GCPN & GraphRNN:**
 - Generate graphs sequentially
 - Imitate a given graph dataset
- **Main Differences:**
 - GCPN uses **GNN** to predict the generation action, rather than **RNN**
 - **Pros:** GNN is more expressive than RNN
 - **Cons:** GNN takes longer time to compute than RNN
 - GCPN further uses **RL** to direct graph generation to our goals
 - RL enables goal-directed graph generation

GCPN vs. GraphRNN

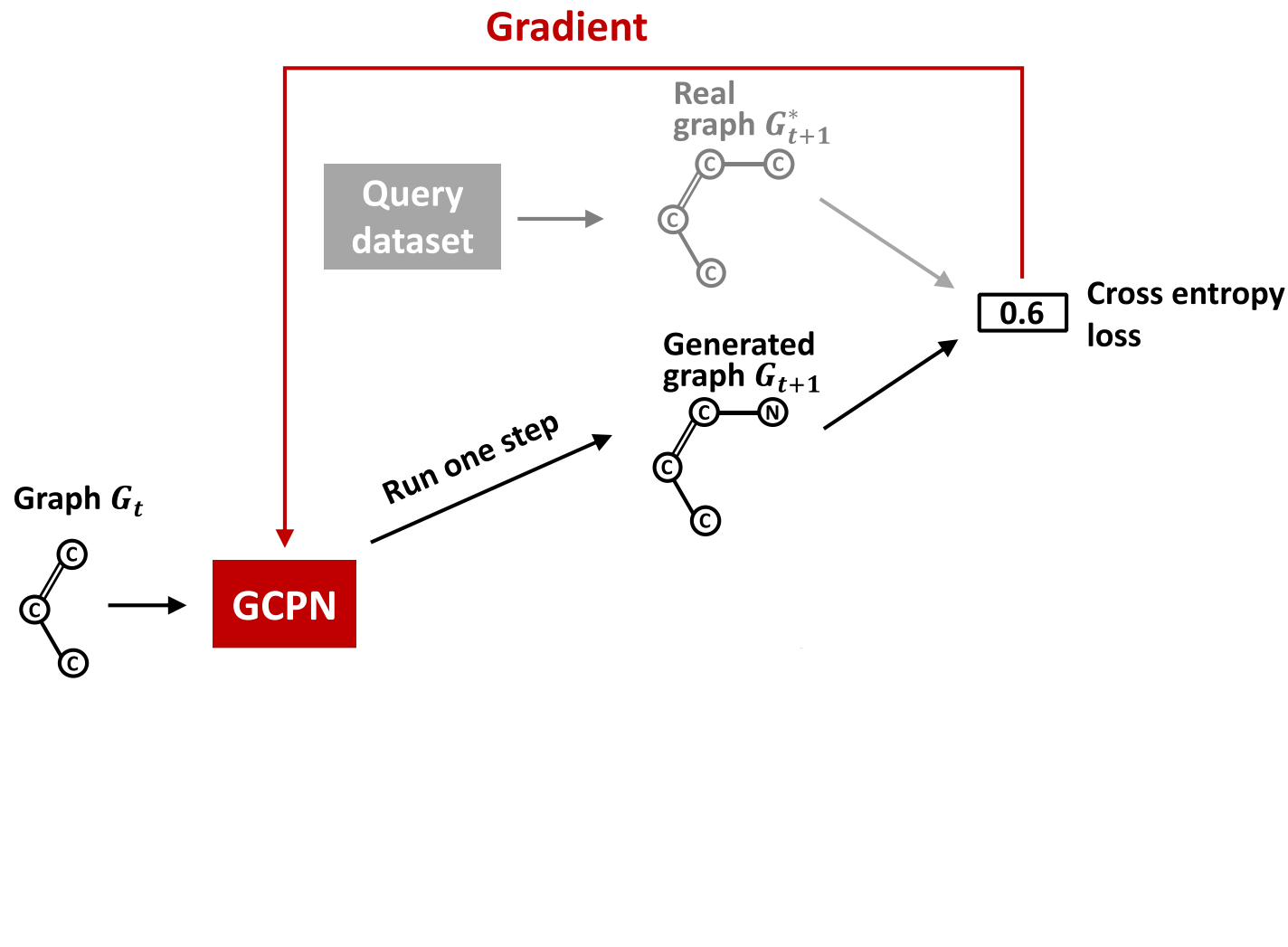
- Sequential graph generation
- GraphRNN: predict action based on **RNN hidden states**



- GCPN: predict action based on **GNN node embeddings**



Training Graph Conv. Policy Network



(1) **Self-supervised training:** Imitating the **action** given by real observed graphs with **gradient**

→ **Goal 1: imitation**

(2) **RL training:** Train policy to **optimize rewards** with **policy gradient (PPO)**

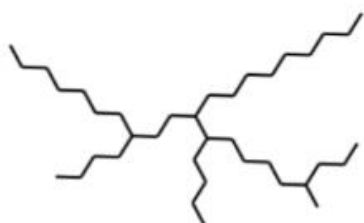
→ **Goal 2: optimization**

ChatGPT [OpenAI, 2022] uses the similar idea: **self-supervised** + **RL training**

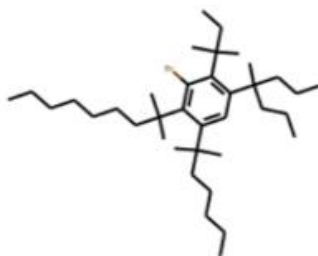
Qualitative Results

Visualization of GCPN graphs:

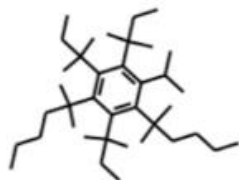
- **Property optimization** Generate molecules with high specified property score



7.98



7.48



7.12

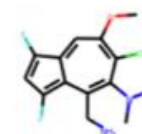


23.88*

(a) Penalized logP optimization



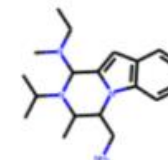
0.948



0.945



0.944



0.941

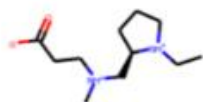
(b) QED optimization

Qualitative Results

Visualization of GCPN graphs:

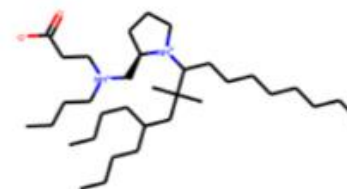
- **Constrained optimization:** Edit a given molecule for a few steps to achieve higher property score

Starting structure

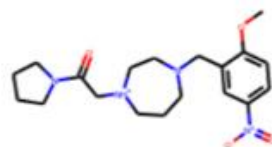


-8.32

Finished structure

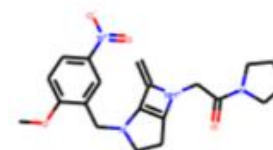


-0.71



-5.55

Increase the
solubility in
octanol



-1.78

(c) Constrained optimization of penalized logP

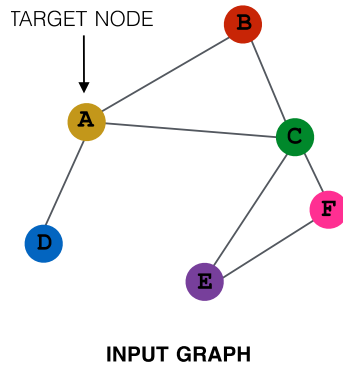
Summary of Graph Generation

- Complex graphs can be successfully generated via autoregressive **sequential generation** using deep learning
- Each step a decision is made based on **hidden state**, which can be
 - **Implicit**: vector representation, decode with RNN
 - **Explicit**: intermediate generated graphs, decode with GCN
- Objectives and tasks:
 - **Imitating** a set of given graphs: **SFT**
 - **Optimizing** graphs towards given goals: **RLHF**
 - 5 years before ChatGPT 😊

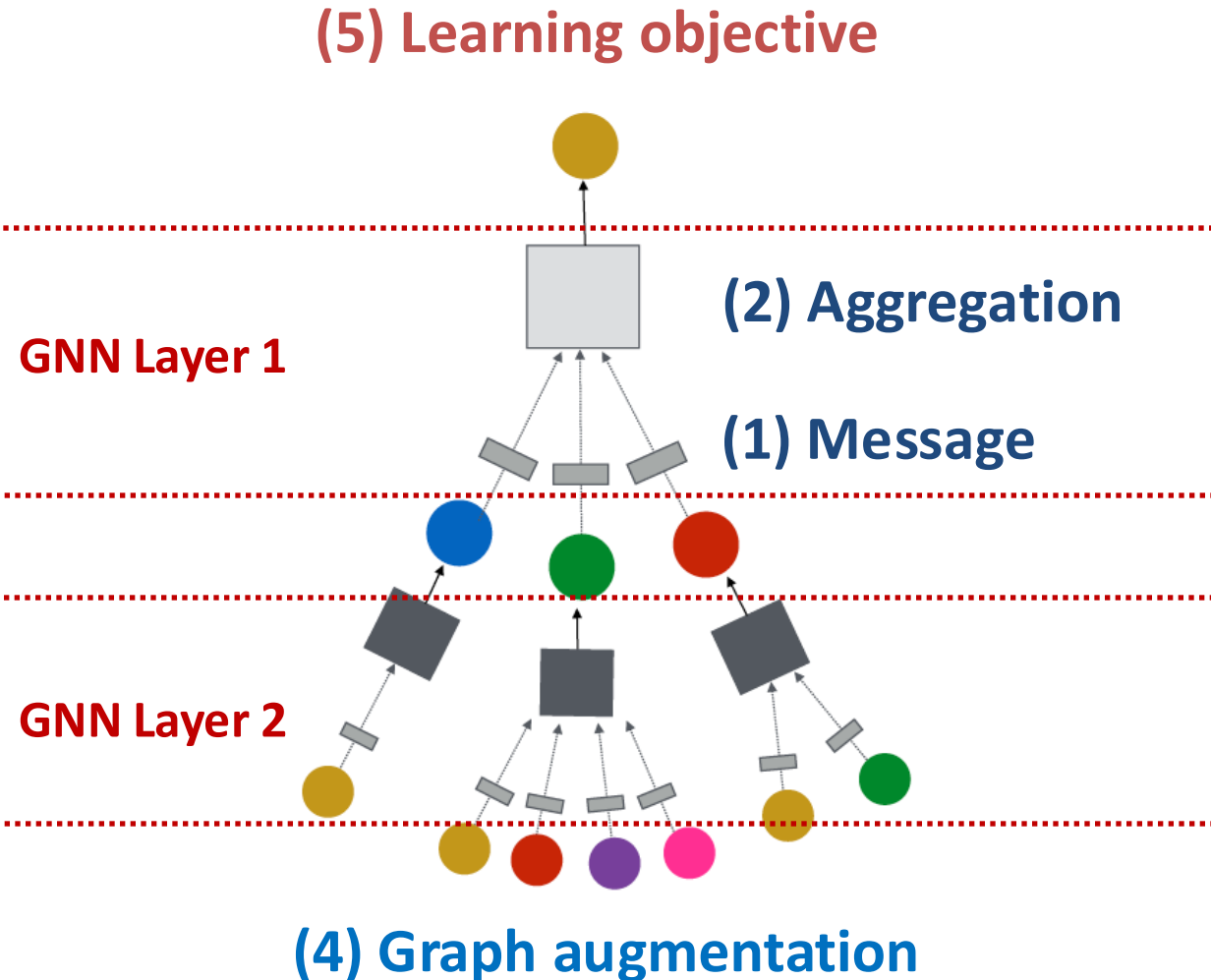
Beyond Message Passing: Expressive GNN Models

Limitations of Graph Neural Networks

Recap: A General GNN Framework

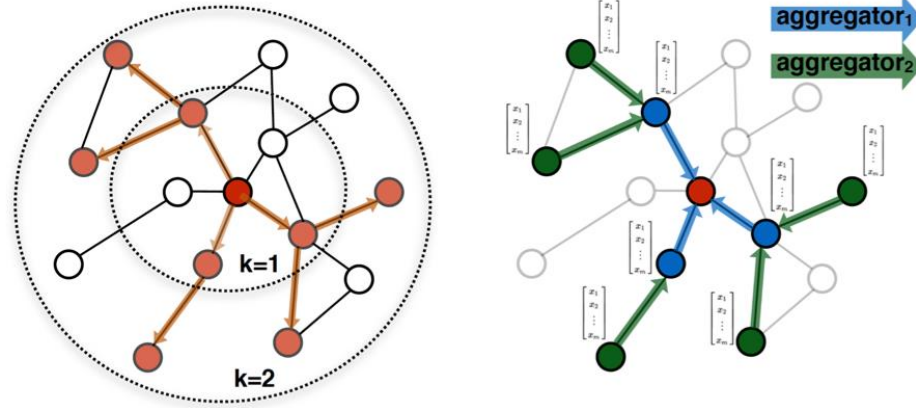


(3) Layer connectivity



A “Perfect” GNN Model

- **A thought experiment:** What should a perfect GNN do?
 - A k -layer GNN embeds a node based on the K -hop neighborhood structure

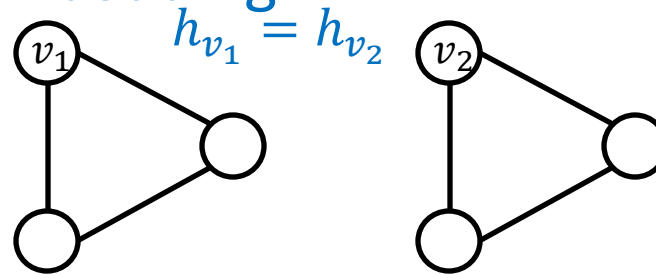


- A perfect GNN should build an **injective function** between **neighborhood structure (regardless of hops)** and **node embeddings**

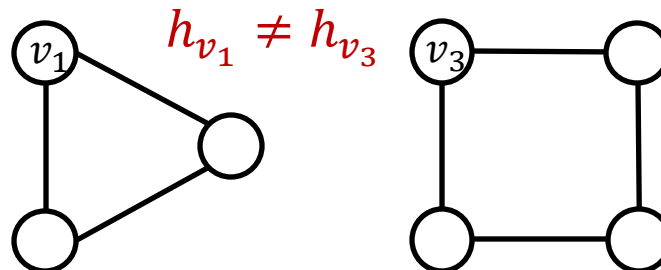
A “Perfect” GNN Model

- Therefore, for a perfect GNN:

- **Observation 1:** If two nodes have the same neighborhood structure, they must have the same embedding

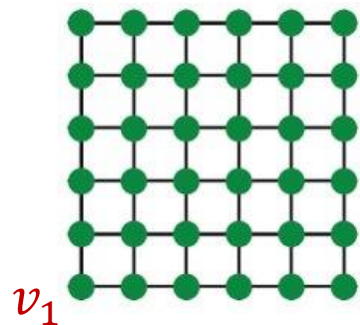


- **Observation 2:** If two nodes have different neighborhood structure, they must have different embeddings

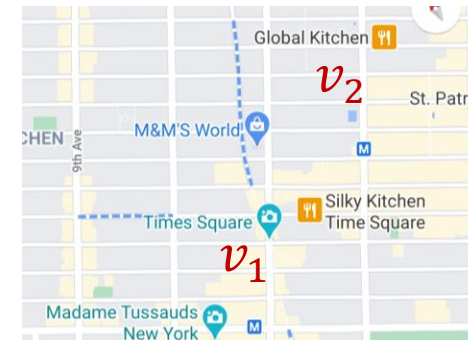


Imperfections of Existing GNNs

- However, Observations **1** & **2** are imperfect
- **Observation 1 could have issues:**
 - Even though two nodes may have the same neighborhood structure, we may want to assign different embeddings to them
 - Because these nodes appear in **different positions in the graph**
 - We call these tasks **Position-aware tasks**
 - **Even a perfect GNN will fail for these tasks:**



A grid graph

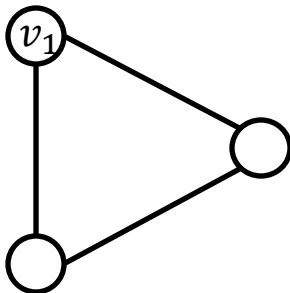


NYC road network

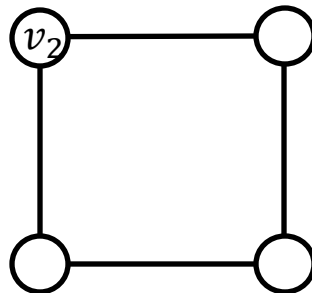
Imperfections of Existing GNNs

- **Observation 2 often cannot be satisfied:**
 - The GNNs we have introduced so far are not perfect
 - In Lecture 9, we discussed that their expressive power is **upper bounded by the WL test**
 - For example, message passing GNNs **cannot count the cycle length:**

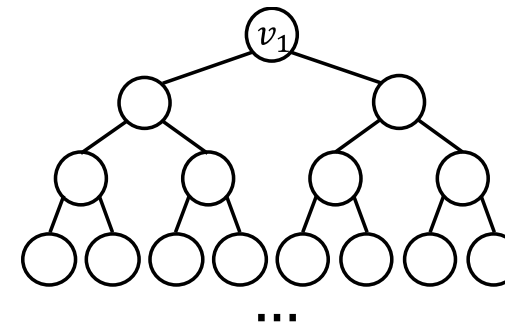
v_1 resides in a cycle with length 3



v_2 resides in a cycle with length 4



The computational graphs for nodes v_1 and v_2 are always the same

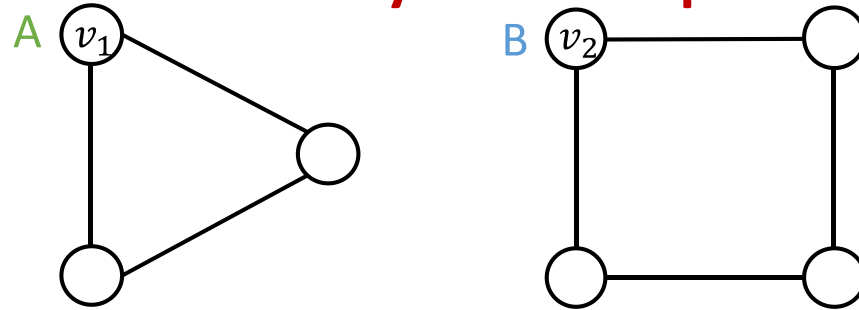


Plan for the Lecture

- We will resolve both issues by **building more expressive GNNs**
- **Fix issues in Observation 1:**
 - Create node embeddings based on their positions in the graph
 - Example method: **Position-aware GNNs**
- **Fix issues in Observation 2:**
 - Build message passing GNNs that are more expressive than WL test
 - Example method: **Identity-aware GNNs**

Our Approach

- We use the following thinking:
 - Two different inputs (nodes, edges, graphs) are labeled differently
 - A “failed” model will always assign the same embedding to them
 - A “successful” model will assign different embeddings to them
 - Embeddings are determined by GNN computational graphs:



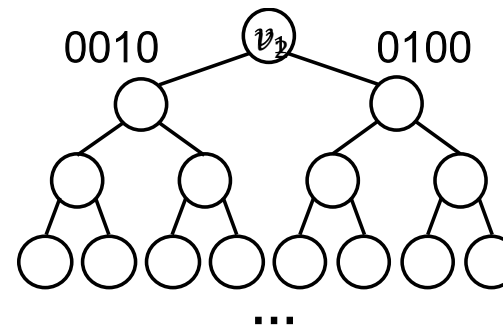
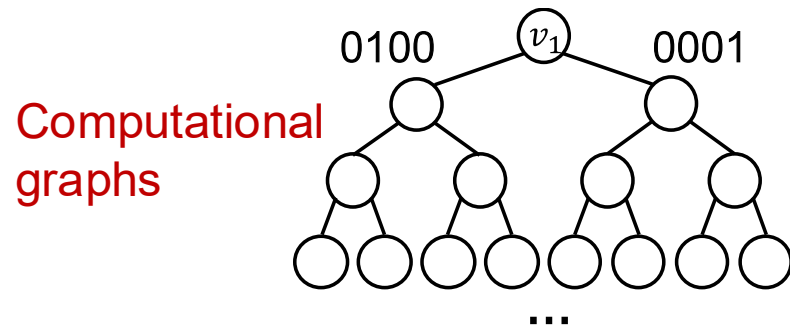
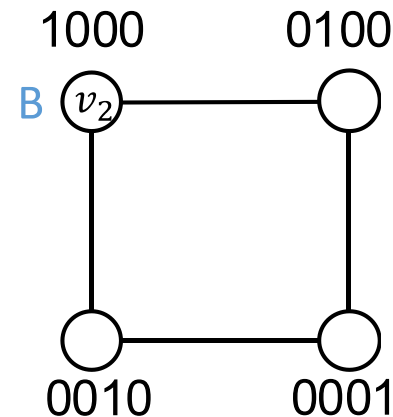
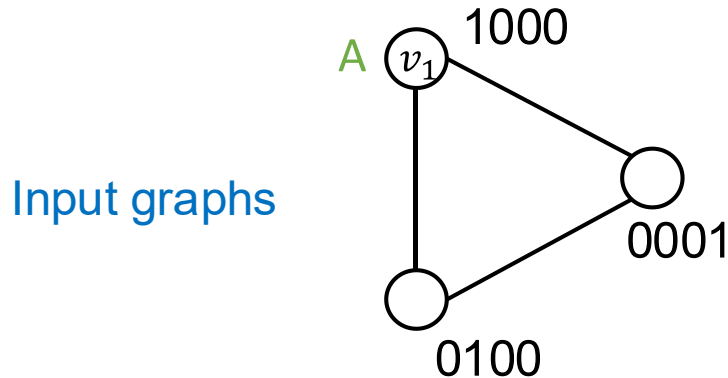
Two inputs: nodes v_1 and v_2

Different labels: A and B

Goal: assign different embeddings to v_1 and v_2

Naïve Solution is not Desirable

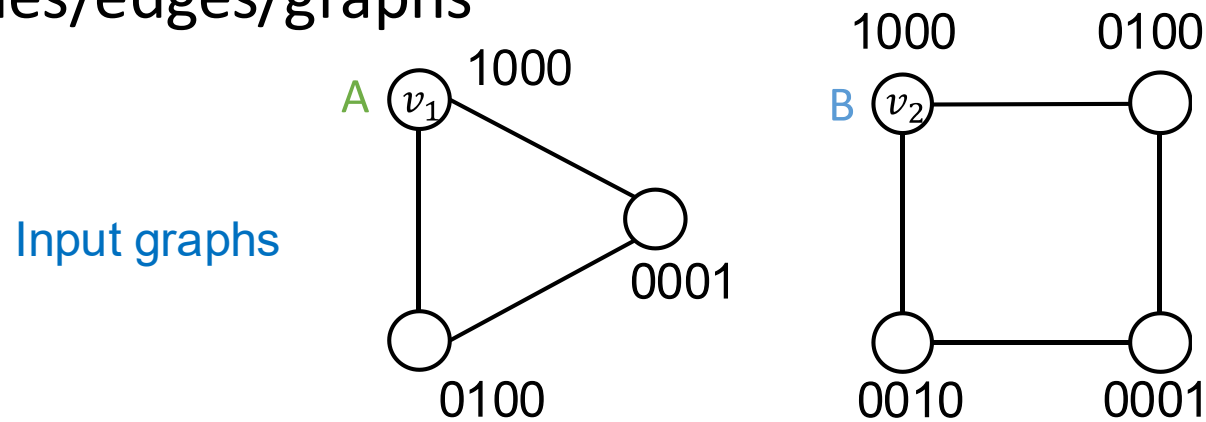
- **A naïve solution:** One-hot encoding
 - Encode each node with a different ID, then we can always differentiate different nodes/edges/graphs



Computational graphs are clearly different if each node has a different ID

Naïve Solution is not Desirable

- **A naïve solution:** One-hot encoding
 - Encode each node with a different ID, then we can always differentiate different nodes/edges/graphs



- **Issues:**
 - **Not scalable:** Need $O(N)$ feature dimensions (N is the number of nodes)
 - **Not inductive:** Cannot generalize to new nodes/graphs

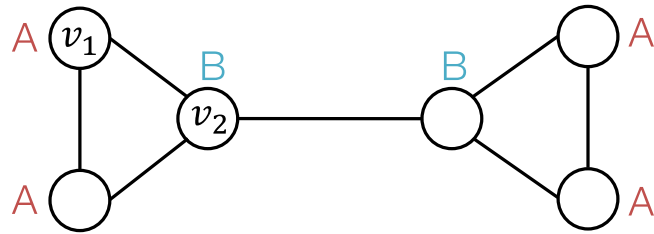
Beyond Message Passing: Expressive GNN Models

Position-aware Graph Neural Networks

Two Types of Tasks on Graphs

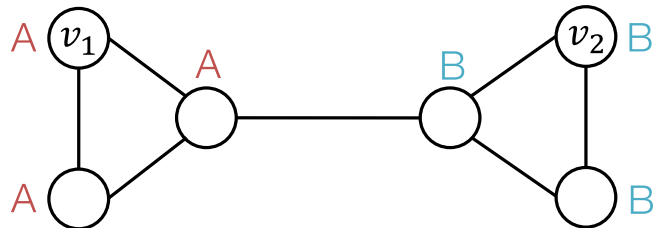
- There are two types of tasks on graphs

Structure-aware task



- Nodes are labeled by their **structural roles** in the graph

Position-aware task

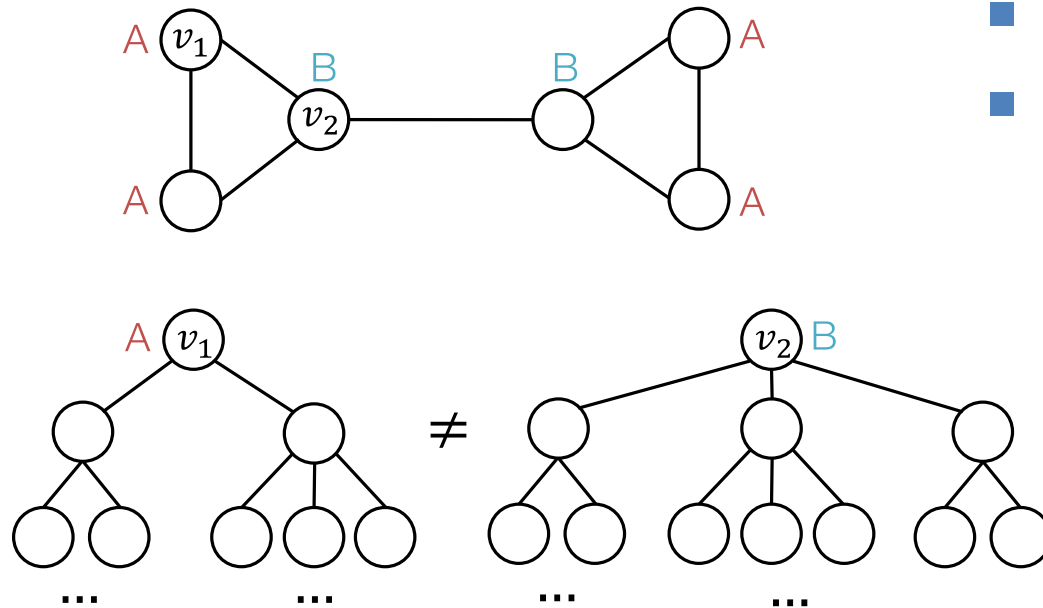


- Nodes are labeled by their **positions** in the graph

Structure-aware Tasks

- GNNs often work well for structure-aware tasks

Structure-aware task

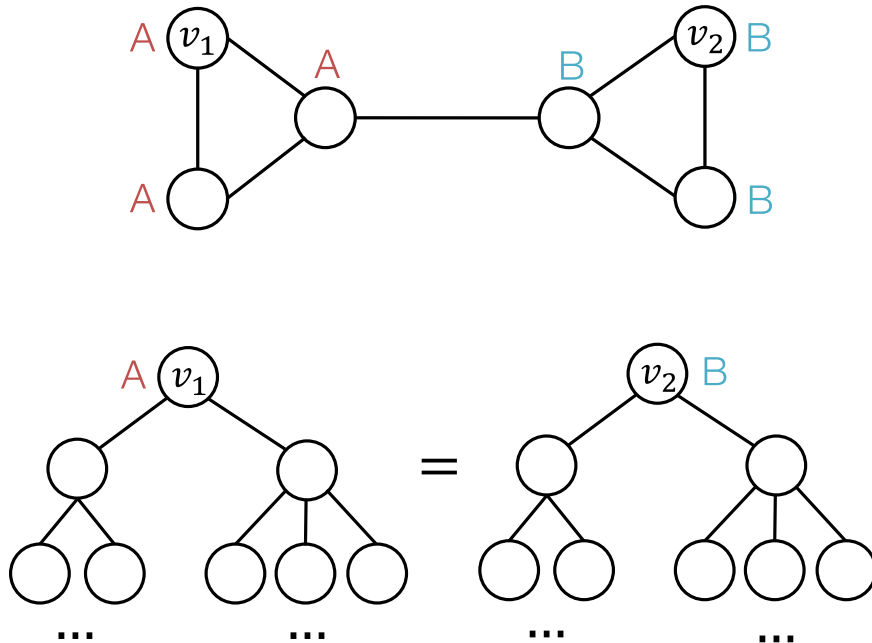


- GNNs work 😊
- Can differentiate v_1 and v_2 by using different computational graphs

Position-aware Tasks

- GNNs will always fail for position-aware tasks

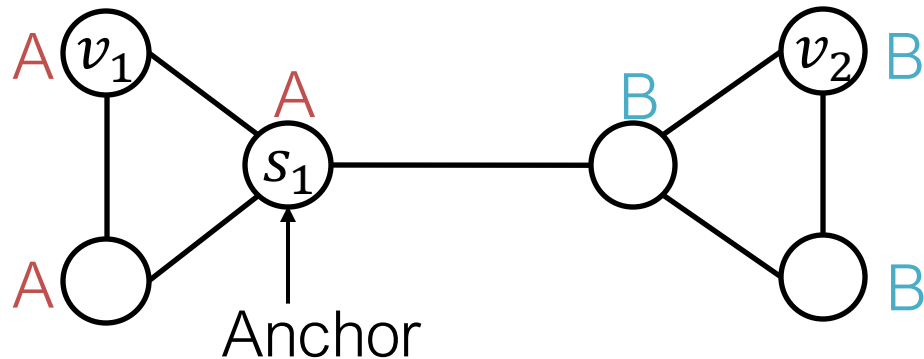
Position-aware task



- GNNs fail 😞
- v_1 and v_2 will always have the same computational graph, **due to structure symmetry**
- Can we define deep learning methods that are position-aware?

Power of “Anchor”

- Randomly pick a node s_1 as an **anchor node**
- Represent v_1 and v_2 via their relative distances w.r.t. the anchor s_1 , **which are different**
- An anchor node serves as **a coordinate axis**
 - Which can be used to **locate nodes in the graph**

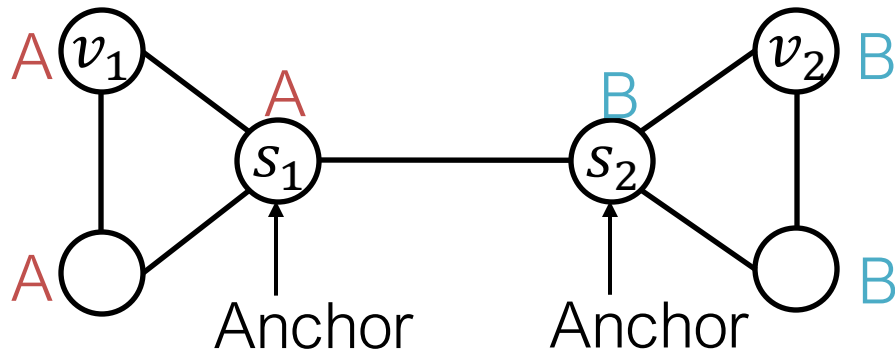


Relative
Distances

	s_1
v_1	1
v_2	2

Power of “Anchors”

- Pick more nodes s_1, s_2 as **anchor nodes**
- **Observation:** More anchors can better characterize node position in different regions of the graph
- Many anchors $\rightarrow$ Many coordinate axes

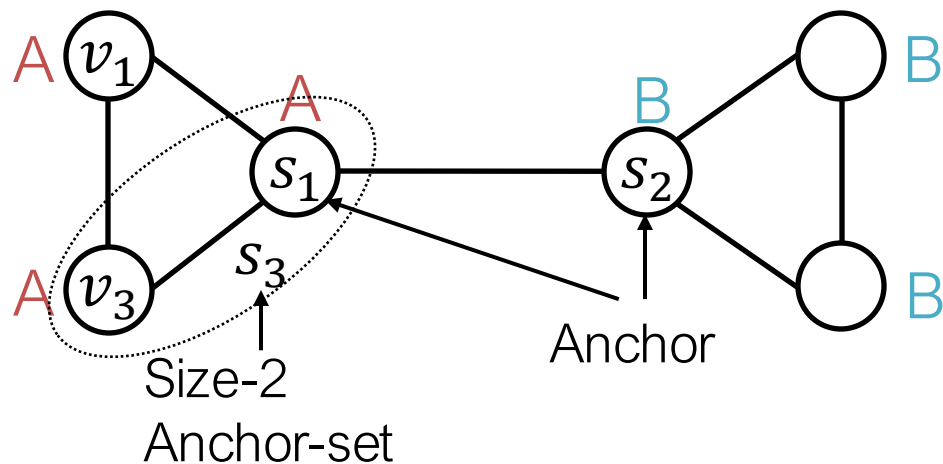


Relative
Distances

	s_1	s_2
v_1	1	2
v_2	2	1

Power of “Anchor-sets”

- Generalize anchor from a single node to a **set of nodes**
 - We define distance to an anchor-set as the minimum distance to all the nodes in the anchor-set
- **Observation:** Large anchor-sets can sometimes provide more precise position estimate
 - We can save the total number of anchors



Relative Distances

	s_1	s_2	s_3
v_1	1	2	1
v_3	1	2	0

Anchor s_1, s_2 cannot differentiate node v_1, v_3 , but anchor-set s_3 can

Anchor Set: Theory

- **Goal:** Embed the metric space (V, d) into the Euclidian space $\mathbb{R}^k$ such that the original distance metric is preserved.
 - For every node pairs $u, v \in V$, the Euclidian embedding distance $\|\mathbf{z}_u - \mathbf{z}_v\|_2$ is close to the original distance metric $d(u, v)$.

Anchor Set: Theory

- Bourgain Theorem [Informal] [Bourgain 1985]
 - Consider the following embedding function of node $v \in V$.
$$f(v) = \left(d_{\min}(v, S_{1,1}), d_{\min}(v, S_{1,2}), \dots, d_{\min}(v, S_{\log n, c \log n}) \right) \in \mathbb{R}^{c \log^2 n}$$
 - where
 - c is a constant.
 - $S_{i,j} \subset V$ is chosen by including each node in V independently with probability $\frac{1}{2^i}$.
 - $d_{\min}(v, S_{i,j}) \equiv \min_{u \in S_{i,j}} d(v, u)$.
 - **The embedding distance produced by f is provably close to the original distance metric (V, d) .**

Anchor Set: Theory

P-GNN follows the theory of Bourgain theorem

- First samples $O(\log^2 n)$ anchor sets $S_{i,j}$.

- Embed each node v via

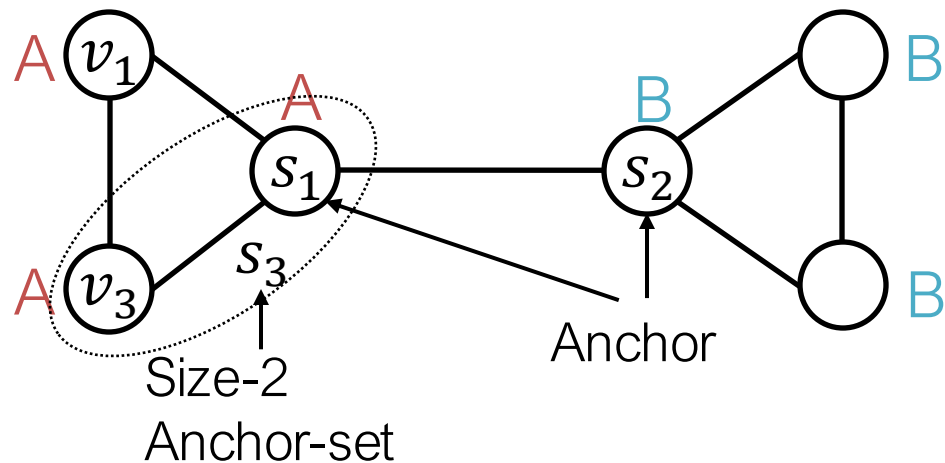
$$\left(d_{\min}(v, S_{1,1}), d_{\min}(v, S_{1,2}), \dots, d_{\min}(v, S_{\log n, c \log n}) \right) \in \mathbb{R}^{c \log^2 n}.$$

P-GNN maintains the inductive capability

- During training, new anchor sets are *re-sampled* every time.
- P-GNN is learned to operate over the new anchor sets.
- At test time, given a new unseen graph, new anchor sets are sampled.

Position Information: Summary

- **Position encoding for graphs:** Represent a node's position by its distance to randomly selected anchor-sets
 - Each dimension of the position encoding is tied to an anchor-set



	s_1	s_2	s_3
v_1	1	2	1
v_3	1	2	0

v_1 's Position encoding

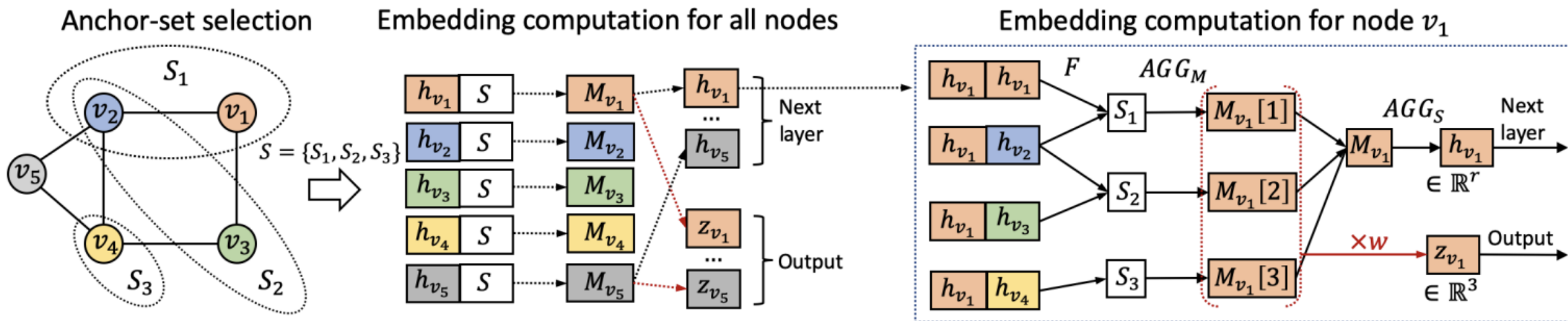
v_3 's Position encoding

How to Use Position Information

- **The simple way:** Use position encoding as an augmented node feature (works well in practice)
 - **Issue:** Since each dimension of position encoding is tied to a random anchor set, dimensions of positional encoding can be randomly permuted, without changing its meaning
 - Imagine you permute the input dimensions of a normal NN, the output will surely change

How to Use Position Information

- **A more expressive solution:** Requires a special NN that can maintain the **permutation invariant property of position encoding**
 - Permuting the input feature dimension will **only result in the permutation of the output dimension**, the value in each dimension won't change
 - Position-aware GNN paper has more details



Beyond Message Passing: Expressive GNN Models

Identity-Aware Graph Neural Networks

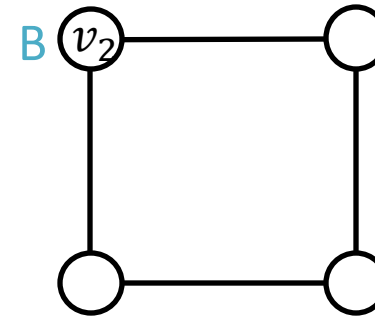
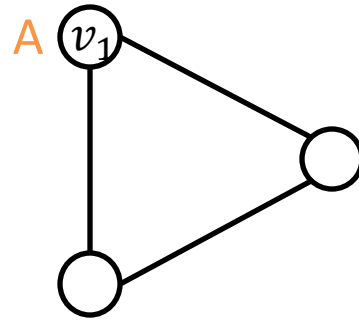
More Failure Cases for GNNs

- We learned that **GNNs would fail for position-aware tasks**
- **But can GNN perform perfectly in structure-aware tasks?**
 - **Unfortunately, NO.**
- GNNs exhibit three levels of failure cases in structure-aware tasks:
 - Node level
 - Edge level
 - Graph level

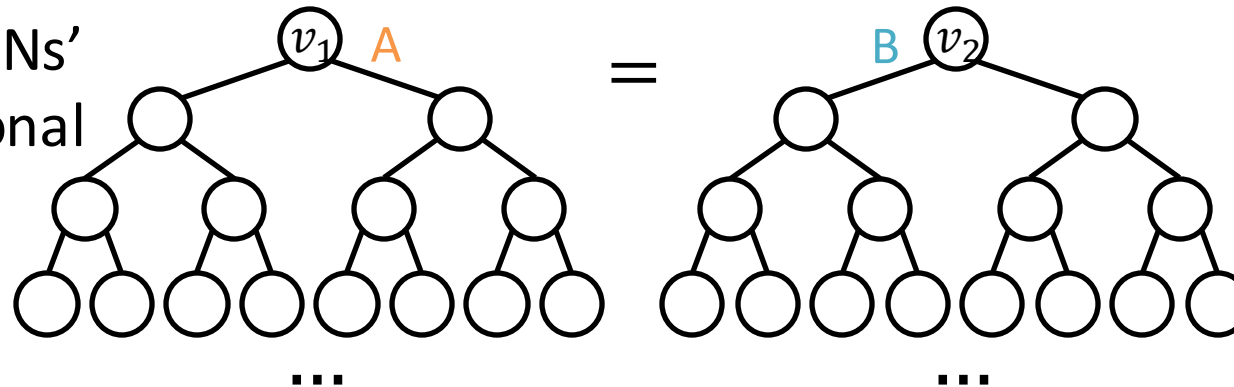
GNN Failure 1: Node-level Tasks

Different Inputs but the same computational graph $\rightarrow$ GNN fails

Example input
graphs



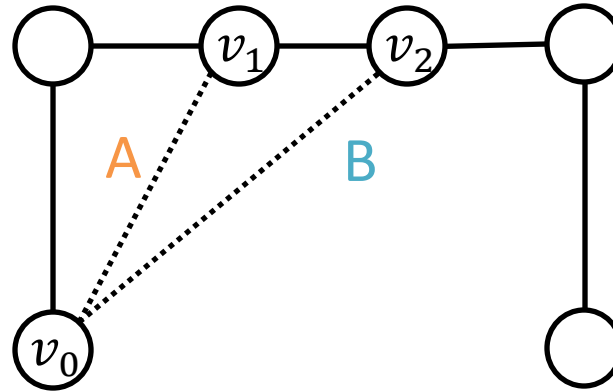
Existing GNNs'
computational
graphs



GNN Failure 2: Edge-level Tasks

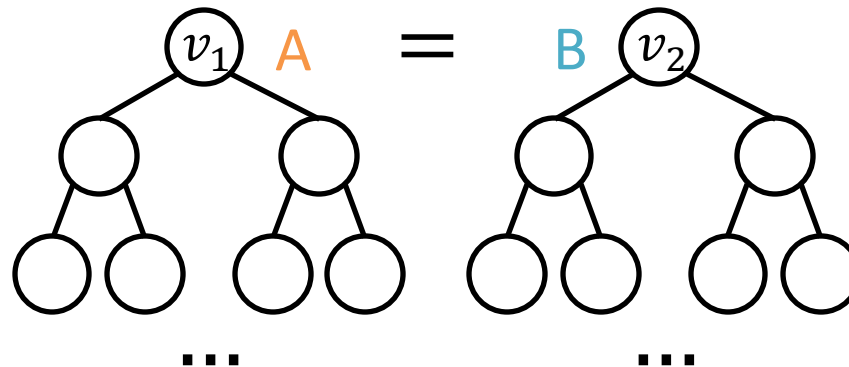
Different Inputs but the same computational graph $\rightarrow$ GNN fails

Example input graphs



Edge **A** and **B** share node v_0
We look at embeddings for v_1 and v_2

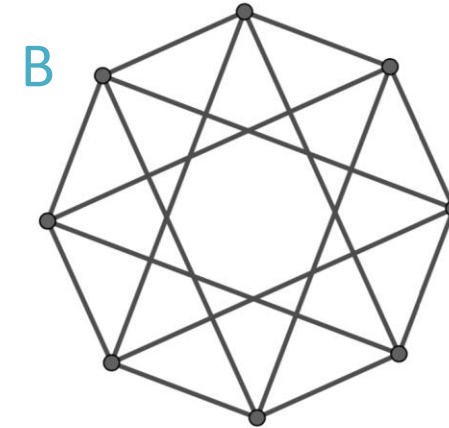
Existing GNNs' computational graphs



GNN Failure 3: Graph-level Tasks

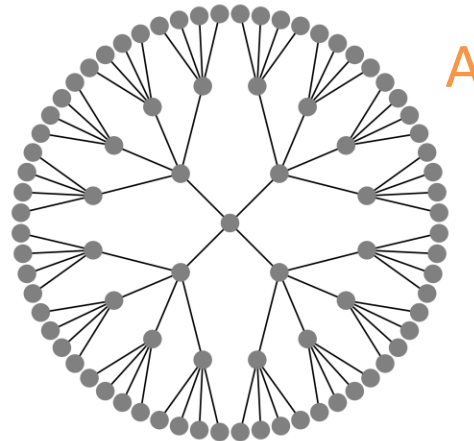
Different Inputs but the same computational graph $\rightarrow$ GNN fails

Example input
graphs

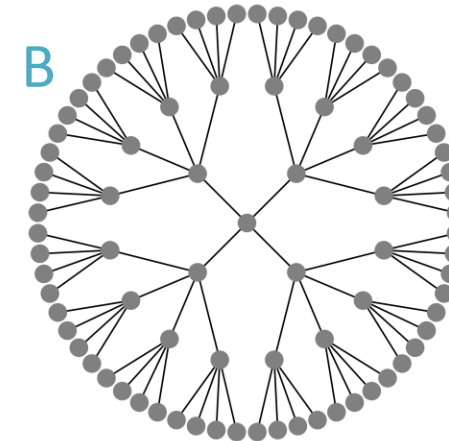


We look at embeddings
for each node

For each node:



For each node:

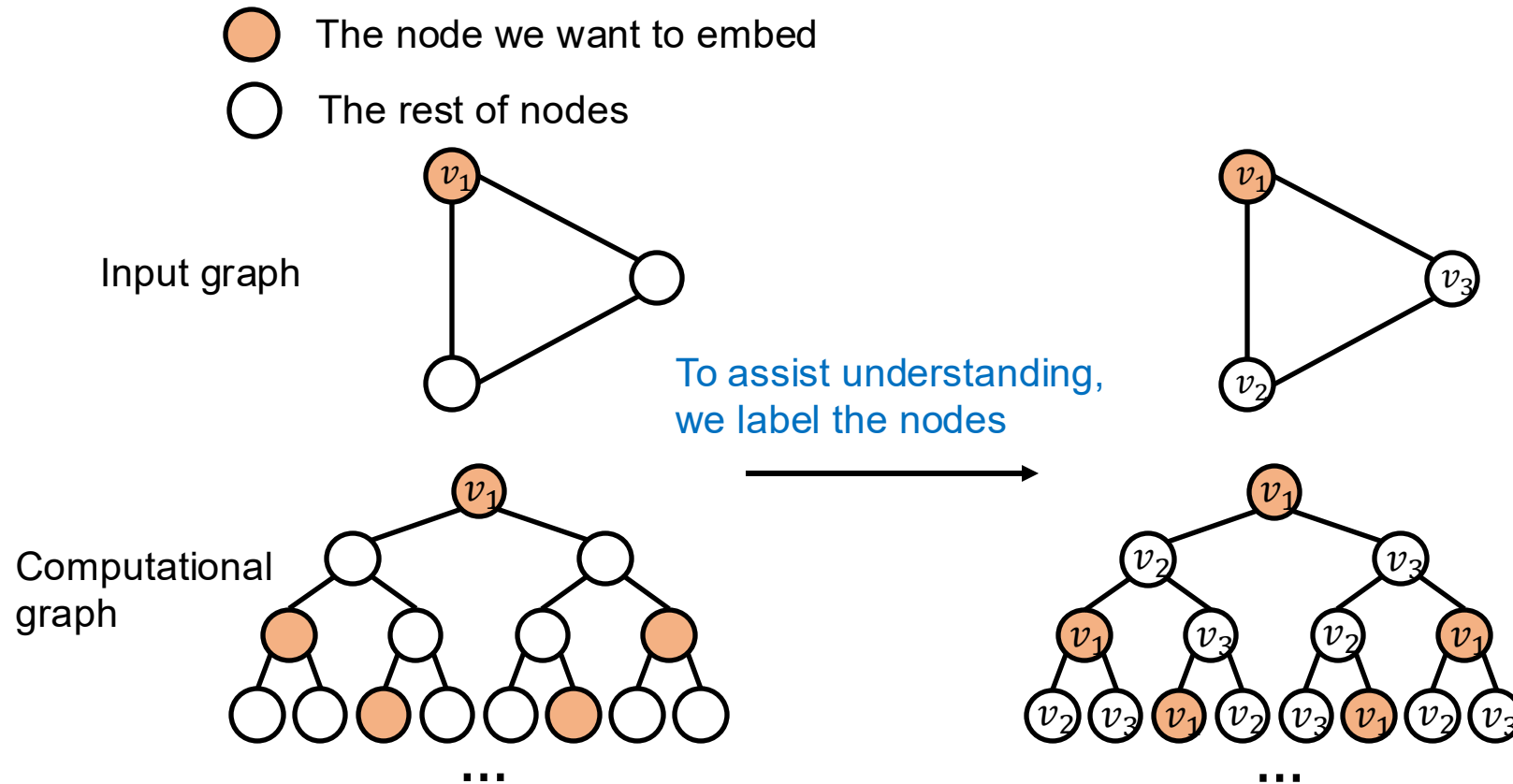


=

Existing GNNs'
computational
graphs

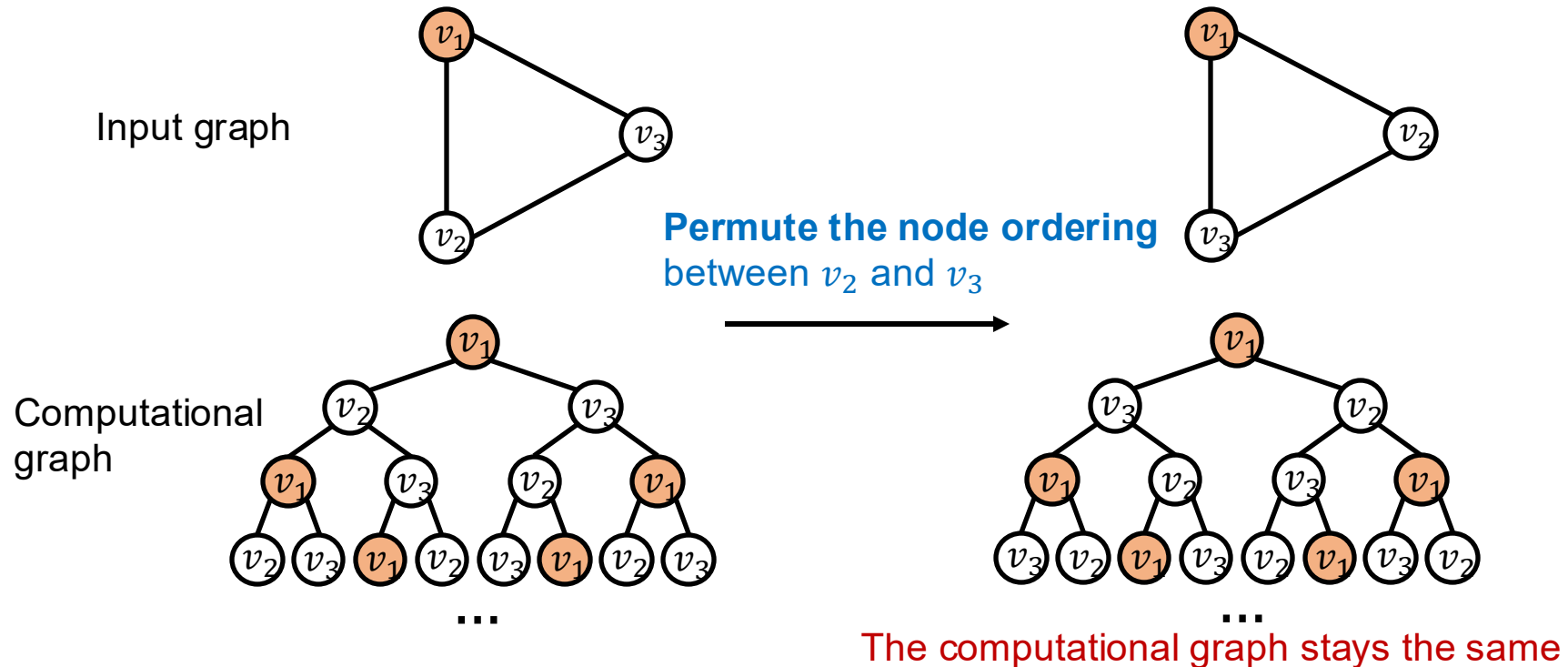
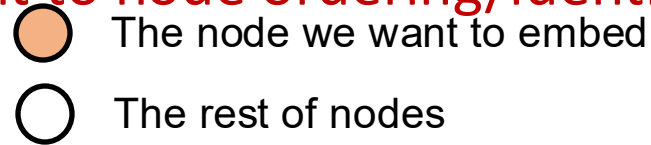
Idea: Inductive Node Coloring

- **Idea:** We can assign a color to the node we want to embed



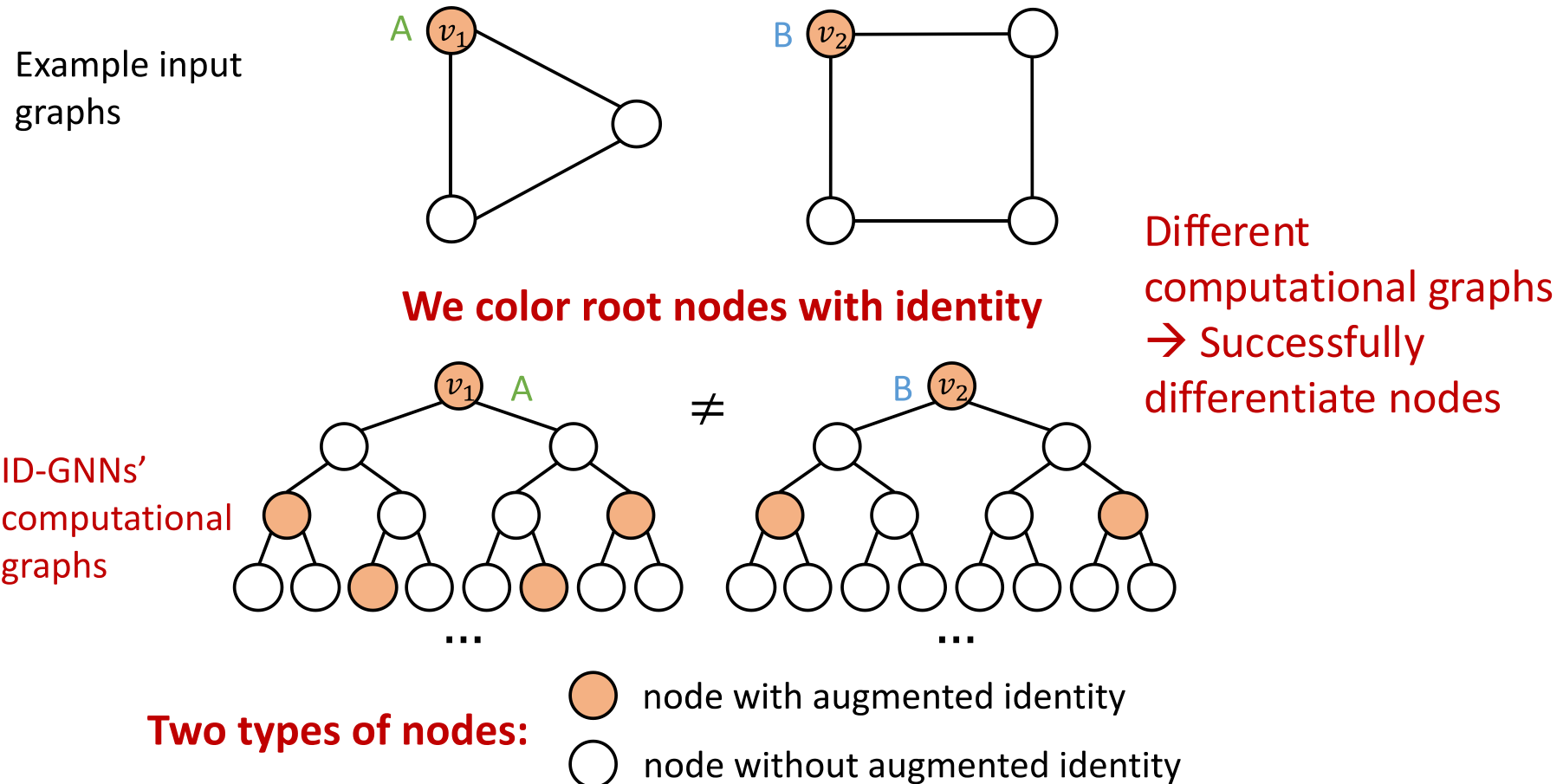
Idea: Inductive Node Coloring

- This coloring is **inductive**:
 - It is **invariant to node ordering/identities**



Inductive Node Coloring – Node level

- Inductive node coloring can help **node classification**

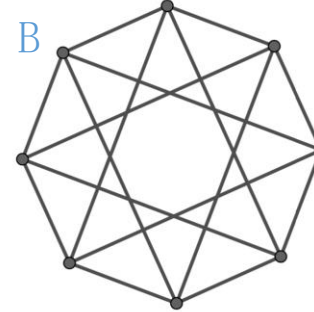


Inductive Node Coloring – Graph Level

- Inductive node coloring can help **graph classification**

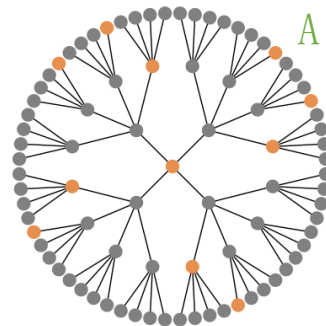
Graph classification

Example input
graphs

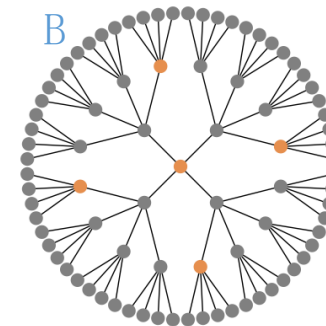


We color root nodes with identity

For each node:



For each node:



$\neq$

Different
computational graphs
→ Successful
differentiate graphs

D-GNNs'
computational
graphs

Two types of nodes:

● node with augmented identity

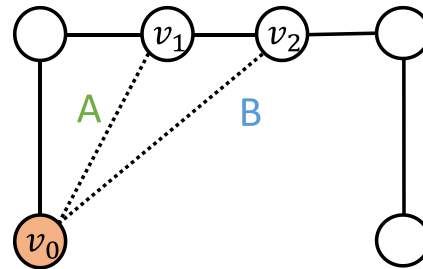
○ node without augmented identity

Inductive Node Coloring – Edge Level

- Inductive node coloring can help **link prediction**

Link prediction

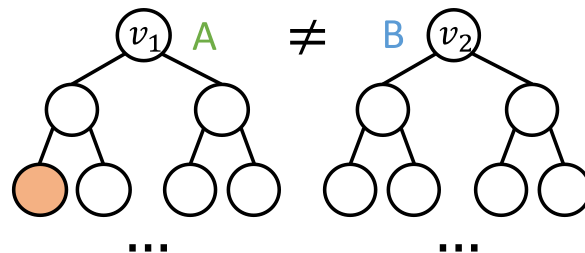
Example input graphs



An edge-level task involves classifying **a pair of nodes**:

- We color one of the node (v_0)
- We then embed the other node in the node pair (v_1 or v_2)
- We use the **node embedding for v_1 or v_2 conditioned on v_0 being colored or not** to make edge-level prediction

ID-GNNs' computational graphs



Two types of nodes:

-  node with augmented identity
-  node without augmented identity

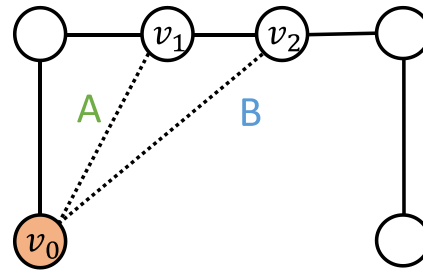
Different computational graphs
→ Successfully differentiate edges

Inductive Node Coloring – Edge Level

- Inductive node coloring can help **link prediction**

Link prediction

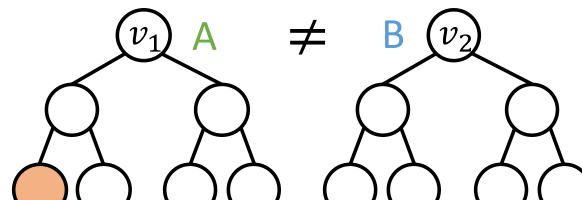
Example input graphs



An edge-level task involves classifying **a pair of nodes**:

- We color one of the node (v_0)
- We then embed the other node in the node pair (v_1 or v_2)
- We use the **node embedding** for v_1 or v_2 conditioned on v_0 being colored or not to make edge-level prediction

ID-GNNs' computational graphs

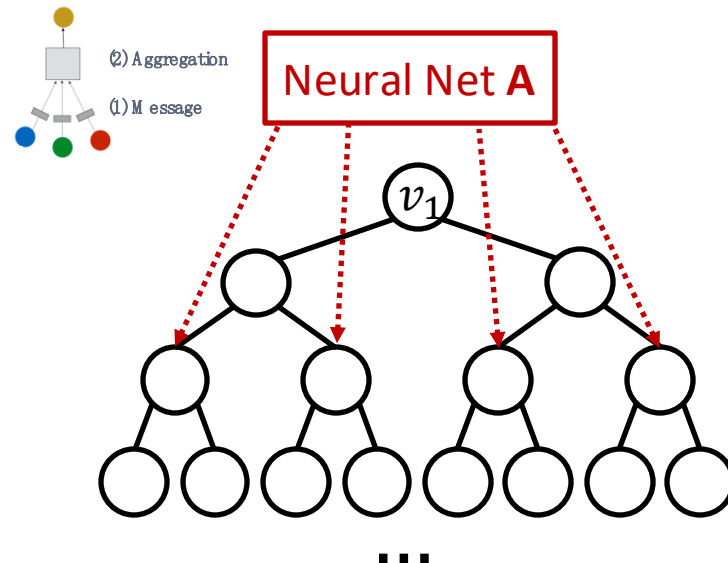


How to build a GNN using node coloring? hs

Two t

Identity-aware GNN

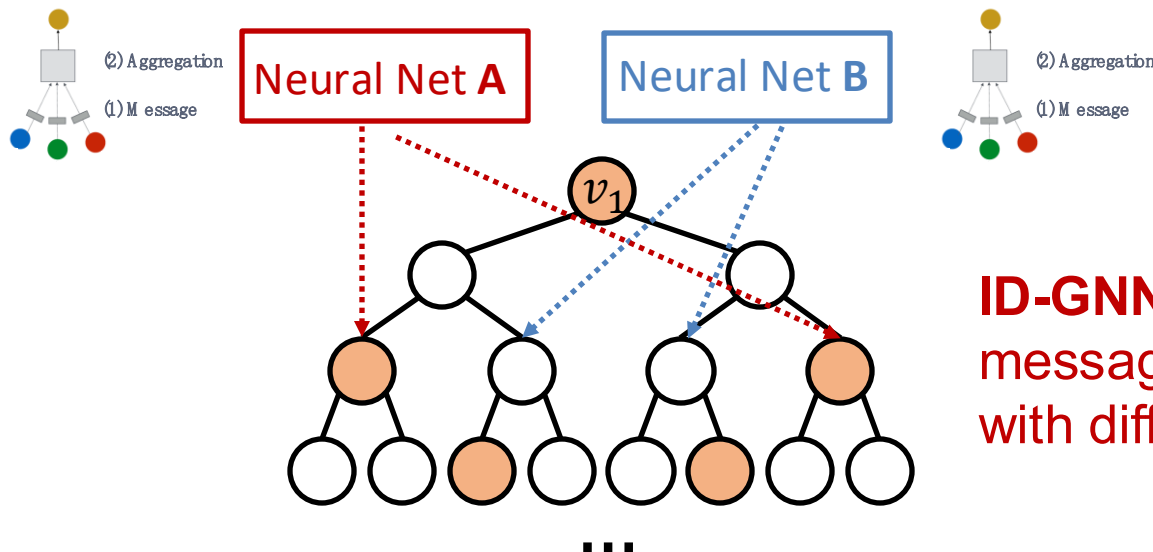
- **Utilize inductive node coloring** in embedding computation
 - **Idea: Heterogenous message passing**
 - Normally, a GNN applies **the same message/aggregation computation to all the nodes**



GNN: At a given layer, we apply the same message/aggregation to each node

Identity-aware GNN

- **Idea: Heterogenous message passing**
 - **Heterogenous:** different types of message passing is applied to different nodes
 - An **ID-GNN** applies **different message/aggregation to nodes with different colorings**



ID-GNN: At a given layer, different message/aggregation to nodes with different colorings

Identity-aware GNN

- **Output:** Node embedding $\mathbf{h}_v^{(K)}$ for $v \in \mathcal{V}$.
- **Step 1:** Extract the ego-network
 - $\mathcal{G}_v^{(K)}$: K -hop neighborhood graph around v
 - Set the initial node feature
 - For $u \in \mathcal{G}_v^{(K)}$, $\mathbf{h}_u^{(0)} \leftarrow \mathbf{x}_u$ (input node feature)

Identity-aware GNN

- **Step 2:** Heterogeneous message passing

- For $k = 1, \dots, K$ do

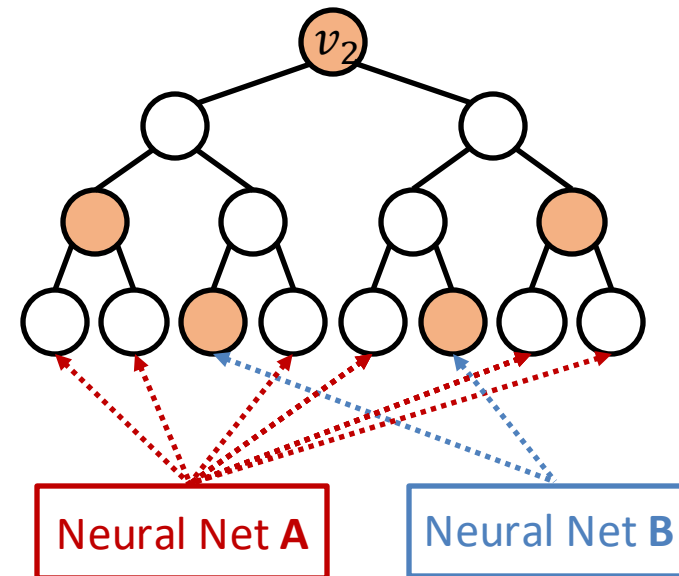
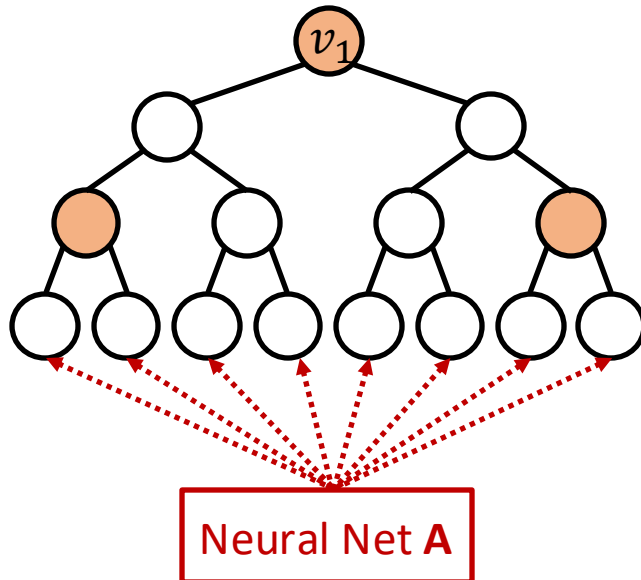
- For $u \in \mathcal{G}_v^{(K)}$ do

- $$\mathbf{h}_u^{(k)} \leftarrow AGG^{(k)} \left(\left\{ MSG_{1[s=v]}^{(k)} \left(\mathbf{h}_s^{(k-1)} \right), s \in N(u) \right\}, \mathbf{h}_u^{(k-1)} \right)$$

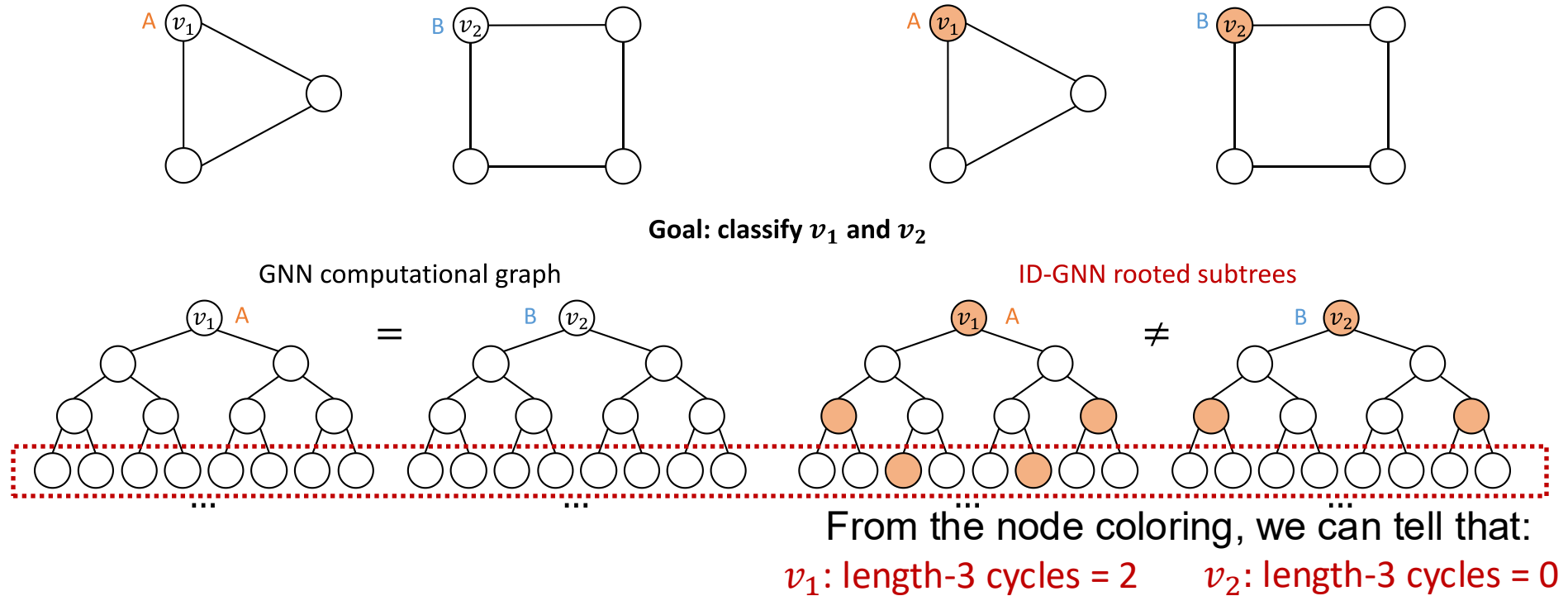
Depending on whether $s = v$ (s is the center node v) or not, we use different neural network functions to transform $\mathbf{h}_s^{(k-1)}$.

Identity-aware GNN

- Why does heterogenous message passing work:
 - Suppose two nodes v_1, v_2 have the same computational graph structure, but have different node colorings
 - Since we will apply different neural network for embedding computation, their embeddings will be different

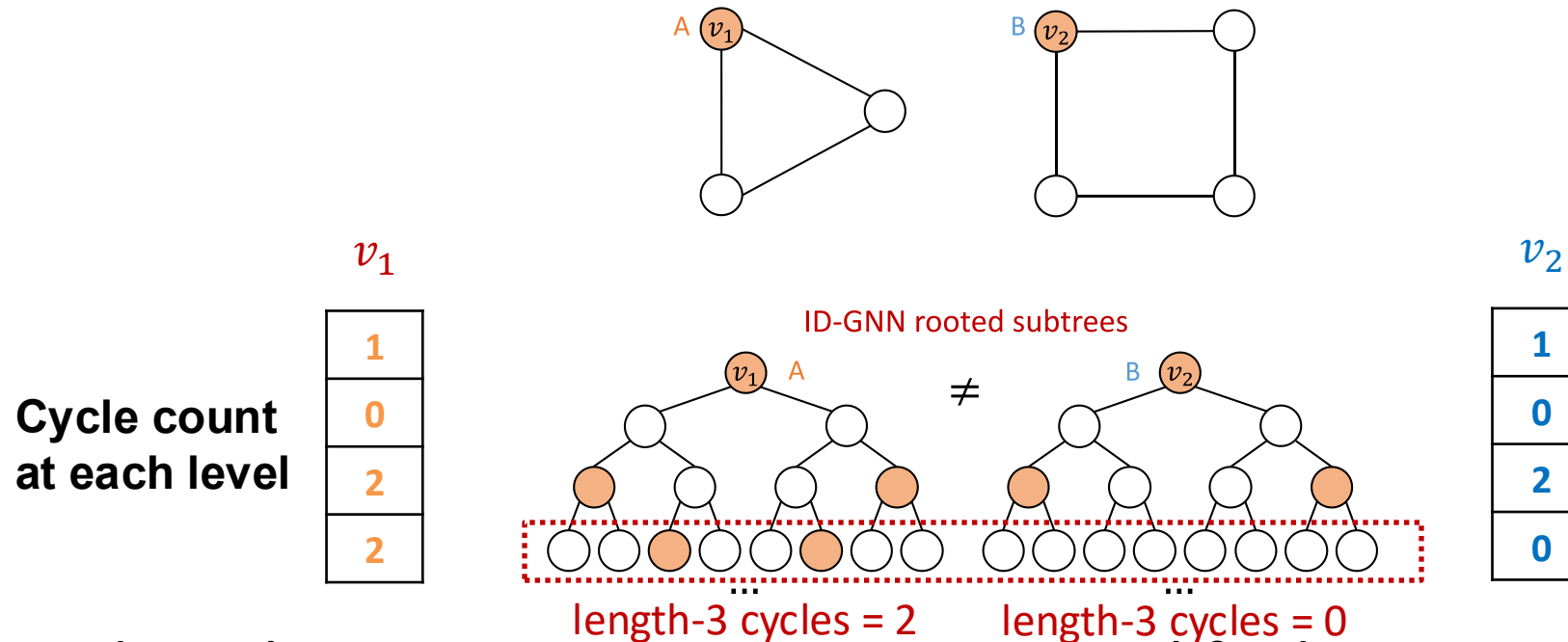


GNN vs. ID-GNN



- Why does ID-GNN work better than GNN?
- Intuition: ID-GNN can count cycles originating from a given node, but GNN cannot

Simplified Version: ID-GNN-Fast



- Based on the intuition, we present a simplified version **ID-GNN-Fast**
 - Include identity information as an **augmented node feature** (no need to do heterogenous message passing)
 - Use cycle counts in each layer as an augmented node feature.** Also can be used together with **any GNN**

Identity-aware GNN

- **Summary of ID-GNN: A general and powerful extension to GNN framework**
 - We can apply ID-GNN on **any** message passing GNNs (GCN, GraphSAGE, GIN, ...)
 - ID-GNN provides **consistent performance gain** in node/edge/graph level tasks
 - ID-GNN is **more expressive** than their GNN counterparts. ID-GNN is **the first message passing GNN that is more expressive than 1-WL test**
 - We can **easily implement** ID-GNN using popular GNN tools (PyG, DGL, ...)